

EYES:ON U

AI Worker 학습·추론 쉬운 안내서

파인튜닝·증류·모델별 역할·CCTV 추론을 한 문서로 설명

중학교 2학년 기준 설명 · 2026-08-10

학습	CLIP·SOLIDER·Qwen
추론	YOLO → 후보 점수 → 관리자 검토
읽는 방법	앞 설명부터 읽고 코드는 필요할 때 확인

EyesOnU AI Worker

학습부터 추론까지 한 번에 이해하는 쉬운 안내서

작성 기준일: 2026-08-10

이 문서만 읽으면 다음 내용을 알 수 있다.

- 어떤 모델을 왜 사용하는가?
- 모델을 어떻게 학습시키는가?
- Sonnet 선생님에게 배운 내용을 어떻게 저장하는가?
- CCTV 영상을 실제로 어떻게 분석하는가?
- 여러 모델의 답을 어떻게 하나의 후보 점수로 합치는가?
- 중앙 서버에 어떤 결과를 보내는가?

코드는 현재 AI Worker 저장소에 있는 실제 코드의 핵심 부분을 쉽게 설명한 것이다. 문서에 명령어가 적혀 있다고 해서 모든 학습이 이미 끝났다는 뜻은 아니다. 마지막에 실제로 확인된 것과 아직 준비 단계인 것을 따로 표시한다.

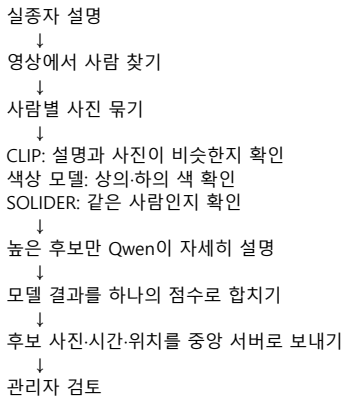
1. 우리가 해결하려는 문제

관리자가 다음과 같이 실종자의 특징을 입력한다고 하자.

“회색 반팔, 검은색 바지, 안경을 쓴 남자. 머리는 뒤로 넘겼다.”

AI Worker는 녹화된 CCTV 영상에서 사람을 찾고, 이 설명과 비슷한 사람을 후보로 뽑는다. 단 한 장의 사진만 보고 바로 “실종자다”라고 확정하지 않는다. 여러 장의 사진, 옷 색, 사람의 생김새, 영상 속 움직임을 함께 보고 후보를 만든 뒤 관리자가 확인한다.

전체 흐름은 다음과 같다.



2. 모델을 사람의 역할로 생각하기

모델을 어려운 수학적식으로만 생각하지 말고, CCTV 조사팀의 여러 담당자라고 생각하면 쉽다.

담당자	사용하는 모델	하는 일	결과
사람 찾기 담당	YOLO	화면 속 사람의 위치를 찾음	사람 상자와 번호
설명 비교 담당	CLIP ViT-L/14	“회색 반팔” 같은 글과 사진을 비교함	비슷한 정도
색상 확인 담당	ROI 색상·속성 head	상의·하의 색을 확인함	색상 점수
사람 구분 담당	SOLIDER	사진 속 사람이 같은 사람인지 비교함	identity 점수

담당자	사용하는 모델	하는 일	결과
최종 설명 담당	Qwen3-VL	상위 후보의 여러 특징을 글로 검토 함	보조 점수와 설명
선생님	Sonnet 5	학습용 속성 답변을 만들어 줌	검수 가능한 라벨

2.1 모델 사이의 관계

YOLO가 먼저 사람을 찾지 못하면 다른 모델은 사람 사진을 받을 수 없다. CLIP·색상 모델·SOLIDER는 각자 다른 증거를 만들고, Qwen은 모든 영상을 처음부터 보는 것이 아니라 이미 좁혀진 상위 후보만 확인한다. 그래서 정확도와 처리 시간을 함께 고려할 수 있다.

Sonnet은 매번 CCTV를 분석하는 모델이 아니다. Sonnet은 학습 전에 사진을 보고 "상의는 회색, 안경은 있음"처럼 정리된 답을 만드는 선생님 역할이다.

Grounding DINO, SAM2.1, Florence-2도 현재 워커가 매번 호출하는 필수 모델이 아니다. 필요하다면 오프라인에서 정확한 상자·마스크·속성 정답을 만드는 데 사용할 수 있다.

3. 먼저 알아둘 쉬운 단어

코드 단어	쉬운 뜻
feature	사진을 숫자로 요약한 값
embedding	사람 사진의 특징을 모은 숫자 지문
head	feature를 보고 색·속성·동일인 여부를 판단하는 작은 층
label	사람이 정해 둔 정답
logit	확률로 바꾸기 전의 원점수
loss	모델의 답이 정답에서 얼마나 틀렸는지 나타내는 점수
optimizer	loss가 작아지도록 모델 숫자를 조금씩 바꾸는 도구
epoch	학습 자료를 한 바퀴 모두 본 횟수
track	영상에서 같은 사람으로 묶은 여러 장의 프레임
teacher	답을 알려주는 선생님 모델
student	teacher의 답을 보며 학습하는 모델
late fusion	여러 모델이 만든 답을 마지막에 합치는 방법

4. 학습은 어떻게 이루어지는가?

모델 학습은 시험공부와 비슷하다.

- 사진과 정답을 준비한다.
- 모델에게 사진을 보여주고 답을 내게 한다.
- 모델 답과 정답을 비교해 틀린 정도를 계산한다.
- 틀린 정도를 줄이도록 모델의 일부 숫자를 바꾼다.
- 이 과정을 여러 번 반복한다.
- 학습에 사용하지 않은 사진으로 진짜 실력을 확인한다.

모델 전체를 항상 바꾸지는 않는다. 이미 잘하는 부분은 고정하고 필요한 부분만 바꾸면 학습이 빠르고, 기존 능력을 잃을 위험도 줄어든다.

학습 코드의 가장 중요한 세 줄은 다음과 같다.

```
loss = answer_error + teacher_help
```

```
loss.backward()    # 어느 방향으로 고쳐야 하는지 계산
optimizer.step()   # 모델 숫자를 실제로 조금 변경
```

loss가 작아진다고 무조건 실제 CCTV에서도 잘 맞는 것은 아니다. 학습 사진과 실제 CCTV의 카메라 각도, 밝기, 거리, 가림 정도가 다르면 별도로 확인해야 한다.

5. CLIP 학습: 글과 사진을 비교하는 담당자

CLIP은 글과 사진을 같은 비교 공간에 놓는 모델이다.

예를 들어 다음 두 문장을 준비할 수 있다.

문장 A: 회색 반팔과 검은색 바지를 입은 사람
문장 B: 빨간 긴팔과 청바지를 입은 사람

사진을 넣었을 때 사진의 특징이 문장 A에 더 가까우면 A의 점수가 높아진다. 하지만 기본 CLIP은 CCTV의 작은 사람, 어두운 화면, 복잡한 배경에서 색과 옷을 충분히 잘 못 볼 수 있다. 그래서 속성 head를 추가로 학습한다.

5.1 CLIP의 속성 head 학습

현재 코드: **scripts/train_clip_vitl14_distill.py**

```
features = _clip_features(clip, pixel_values)
head = BinaryAttributeHead(features.shape[1]).to(device)
optimizer = torch.optim.AdamW(
    head.parameters(), lr=2e-3, weight_decay=1e-4
)

logits = head(features)
hard_loss = _weighted_bce(logits, labels, positive_ratio)

optimizer.zero_grad(set_to_none=True)
hard_loss.backward()
optimizer.step()
```

코드를 쉽게 읽으면 다음과 같다.

- **features**: CLIP이 사진을 숫자로 요약한다.
- **head**: 그 숫자를 보고 여러 속성의 답을 낸다.
- **labels**: 실제 정답이다. 예를 들어 회색이면 1, 아니면 0이다.
- **_weighted_bce**: 자주 나오지 않는 속성도 무시하지 않도록 틀린 정도를 조정한다.
- **backward()**: 어디가 틀렸는지 계산한다.
- **step()**: head의 숫자를 고친다.

CLIP 본체를 고정하고 작은 head부터 학습하는 이유는 이미 배운 일반적인 사진 이해 능력을 보존하면서 우리 속성에 맞게 빠르게 조정하기 위해서다.

5.2 SOLIDER의 점수를 CLIP이 배우게 하기

SOLIDER가 같은 사진을 보고 낸 속성 점수가 있다면, CLIP head가 그 점수도 참고하게 할 수 있다. 이것이 logit 증류다.

```
hard_loss = _weighted_bce(logits, labels, positive_ratio)

if teacher_logits is not None and distill_alpha > 0:
    soft_target = (teacher_logits / temperature).sigmoid()
    soft_loss = F.binary_cross_entropy_with_logits(
        logits / temperature,
        soft_target,
    ) * temperature**2
    loss = (1 - distill_alpha) * hard_loss + distill_alpha * soft_loss
else:
    loss = hard_loss

optimizer.zero_grad(set_to_none=True)
loss.backward()
optimizer.step()
```

기본 실험값은 다음과 같다.

```
distill_alpha = 0.35 # 선생님 점수를 반영하는 비율
temperature = 2.0   # 선생님 점수 차이를 부드럽게 만드는 값
```

쉽게 말하면 정답을 직접 맞히는 힘을 65%, SOLIDER 선생님을 따라가는 힘을 35%로 둔 것이다. 여기서 **teacher_logits**는 Sonnet의 내부 점수가 아니라 SOLIDER 속성 head의 점수다.

PA-100K에는 같은 사람을 여러 카메라에서 찾는 identity-track 정답이 없으므로, 이 학습 결과는 CCTV 동일인 정확도와 같은 뜻이 아니다.

6. Sonnet 선생님을 이용한 속성 학습

Sonnet은 학생 모델의 가중치를 직접 보내주는 것이 아니다. 사진을 보고 사람이 읽을 수 있는 답을 만들어 준다.

예를 들어 Sonnet의 답을 다음처럼 정리할 수 있다.

```
{
  "decision": "review",
  "attributes": {
    "upper_color": "gray",
    "lower_color": "black",
    "glasses": true,
    "hair": "slicked_back"
  },
  "confidence": 0.72
}
```

이 답은 사람이 검수한 뒤 학생 모델의 속성 정답으로 사용한다.

6.1 Sonnet 답변을 학습 기록으로 저장하기

현재 코드: **src/qwen_backend/annotation_cli.py**

```
uv run python -m qwen_backend.annotation_cli \
  --image datasets/candidate/images/cam01/000001.jpg \
  --image-root datasets/candidate/images \
  --sample-id cam01-000001 \
  --teacher-model claude-sonnet-5 \
  --source-kind sonnet \
  --prompt-version sonnet-candidate-v1 \
  --approval-status approved \
  --reviewed-by operator-001 \
  --decision review \
  --confidence 0.72 \
  --color gray \
  --clothing shirt \
  --object-name person \
  --track-id 17 \
  --output datasets/candidate/distillation.jsonl
```

각 옵션의 뜻은 다음과 같다.

옵션	뜻
--image	분석할 사진
--teacher-model	답을 만든 선생님 이름
--source-kind	답의 출처. 여기서는 Sonnet
--prompt-version	Sonnet에게 사용한 질문 버전
--approval-status	사람이 검수했는지
--reviewed-by	검수자 이름
--confidence	선생님 답변의 확신 정도
--output	학습 기록을 저장할 JSONL 파일

프로그램은 사진의 SHA-256 지문도 함께 저장한다. 나중에 사진이 바뀌면 지문이 달라지므로 학습 전에 알아낼 수 있다.

6.2 승인되지 않은 답은 학습에 넣지 않기

현재 코드: `src/qwen_backend/distillation.py`

```
samples = read_distillation_samples(input_jsonl)
records = tuple(
    to_qwen_record(
        sample,
        image_root,
        verify_hash=True,
    )
    for sample in samples
)
write_qwen_jsonl(records, output_jsonl)
```

이 코드는 다음을 확인한다.

- 사진이 허용된 폴더 안에 있는가?
- 저장된 사진 지문과 현재 사진 지문이 같은가?
- 사람이 승인했거나 teacher agreement가 있는가?
- teacher 이름과 질문 버전이 허용 목록에 있는가?

하나라도 틀리면 Qwen 학습 자료로 바꾸지 않는다. 이것은 선생님이 잘못 답한 내용이 학생에게 그대로 퍼지는 것을 막는 안전장치다.

7. SOLIDER 학습: 같은 사람인지 비교하는 담당자

SOLIDER는 “이 사진의 사람과 저 사진의 사람이 같은 사람인가?”를 잘 판단하도록 만드는 ReID 모델이다. 얼굴만 보는 것이 아니라 옷, 가방, 몸의 모양 같은 전체 특징을 숫자 지문으로 만든다.

현재 코드: `scripts/finetune_prid2011_solider_backbone.py`

기존 SOLIDER 선생님은 고정하고 학생의 마지막 부분만 학습한다. 학습할 때 한 batch 안에 같은 사람 사진과 다른 사람 사진을 같이 넣는다.

```
arc_loss = F.cross_entropy(
    arc_head(features, labels),
    labels,
    label_smoothing=0.10,
)
triplet_loss = batch_hard_triplet(features, labels, margin=0.20)
local_loss = part_triplet(final_map, labels, parts=4, margin=0.20)
preservation_loss = 1.0 - F.cosine_similarity(
    features,
    teacher_features,
).mean()

loss = (
    arc_loss
    + triplet_weight * triplet_loss
    + part_weight * local_loss
    + teacher_weight * preservation_loss
)

scaler.scale(loss).backward()
scaler.step(optimizer)
scaler.update()
```

각 학습 항을 쉽게 설명하면 다음과 같다.

- **arc_loss**: 같은 사람과 다른 사람 사이에 더 넓은 간격을 만든다.
- **triplet_loss**: 가장 헛갈리는 같은 사람·다른 사람 사진을 골라 비교한다.
- **local_loss**: 사진을 가로 네 부분으로 나눠 모자·상의·하의·신발 같은 세부 정보도 본다.
- **preservation_loss**: 파인튜닝 전 SOLIDER가 잘하던 것을 잃지 않게 한다.

scaler는 GPU에서 숫자를 조금 작게 바꿔 계산하는 도구다. 계산 속도와 메모리 사용량을 줄이면서 학습할 수 있다.

8. Qwen 증류: 사진을 보고 설명하는 담당자

Qwen은 후보 사진과 실종자 설명을 함께 보고 다음과 같은 답을 만들 수 있다.

```
{
  "decision": "review",
  "attributes": {
    "upper_color": "gray",
    "lower_color": "black",
    "glasses": "uncertain"
  },
  "reason": "상의와 하의 색은 비슷하지만 안경은 화면이 작아 확인이 어렵다.",
  "confidence": 0.68
}
```

Qwen 학습을 위한 한 줄은 "사진 + 질문 + JSON 답변"으로 만든다.

```
uv run python -m qwen_backend.distillation_cli validate W
--input datasets/candidate/distillation.jsonl W
--image-root datasets/candidate/images

uv run python -m qwen_backend.distillation_cli prepare W
--input datasets/candidate/distillation.jsonl W
--image-root datasets/candidate/images W
--output datasets/candidate/qwen_train.jsonl
```

validate는 학습 자료가 규칙에 맞는지 확인한다. **prepare**는 Qwen이 읽을 수 있는 형식으로 변환한다.

8.1 Qwen LoRA 학습

LoRA는 Qwen 전체를 다시 고치는 대신 작은 추가 층만 학습하는 방법이다. 그래서 GPU 메모리와 저장 공간을 아낄 수 있다.

현재 저장소에는 Qwen 장시간 학습 파일 **train_qwen_lora.sh**가 없다. 따라서 다음은 GPU 서버에 runner가 준비된 경우의 실행 형식이다.

```
cd /home/j-i15a204/qwen3vl-backend

# 먼저 짧은 동작 확인
DRY_RUN=1 bash training/train_qwen_lora.sh

# 실제 실행 형식
DRY_RUN=0 NPROC_PER_NODE=4 bash training/train_qwen_lora.sh W
2>&1 | tee /home/j-i15a204/outputs/qwen3vl-train.log
```

실제 모델로 채택하려면 다음 자료가 모두 있어야 한다.

- 짧은 smoke test 결과
- validation-test 결과
- 저장된 checkpoint의 SHA-256
- GPU 학습 로그
- 어떤 데이터와 설정을 사용했는지 적은 manifest

이 자료가 회수되기 전에는 "Qwen 학습이 끝났다"고 말하지 않는다.

9. CCTV 추론은 어떻게 실행되는가?

학습은 미리 하는 일이고, 추론은 실제 영상이 들어왔을 때 하는 일이다.

9.1 워커가 받는 요청

현재 코드: **src/qwen_backend/candidate_runtime.py**

워커는 중앙 서버에서 다음 정보를 받는다.

```
{
  "schemaVersion": "eyesonu-candidate-runtime-v1",
  "modelKey": "hybrid-solider-clip-v1",
  "jobId": 70,
  "caseId": 12,
  "recordingId": 301,
  "cameralId": 11,
  "cameraName": "1-1",
  "cameraAddress": "zone-1-camera-1",
  "videoPath": "C:/worker/input/segment.mp4",
  "referencePath": "C:/worker/input/reference.jpg",
  "outputDir": "C:/worker/output/job-70",
  "prompt": "회색 반팔, 검은색 바지, 안경을 쓴 남자",
  "searchFromMs": 0,
  "searchToMs": 60000
}
```

similarityThreshold가 비어 있으면 서버가 자동 컷오프를 정하지 않고 후보를 순위순으로 받겠다는 뜻이다. 워커는 그래도 색상 불일치, 너무 낮은 후보 점수 같은 기본 안전 규칙은 적용한다.

9.2 모델을 미리 준비하기

MultiModelCandidateEngine.warm_up()은 첫 영상이 들어오기 전에 모델을 준비하고 캐시한다.

```
def warm_up(self) -> None:
    self._get_clip_bundle()
    self._get_fine(self._get_clip_bundle())
    self._get_solider_par()
    self._qwen_review.warm_up()
```

모델을 매 프레임마다 다시 읽으면 매우 느리다. 그래서 프로그램을 켜 때 한 번 읽고, 이후 영상에서는 이미 메모리에 있는 모델을 사용한다. Qwen은 설정되어 있을 때만 준비한다.

9.3 YOLO가 사람을 찾고 사진을 저장하기

현재 코드: **src/qwen_backend/video_tracks.py**

```
results = model.track(
    source=str(video_path),
    stream=True,
    classes=[0], # 사람 클래스만 사용
    conf=confidence,
    tracker=tracker,
    device=device,
    vid_stride=stride,
    persist=True,
    verbose=False,
)
```

YOLO는 화면 전체에서 사람의 네모 상자를 찾는다. **classes=[0]**은 자동차나 간판이 아니라 사람만 받겠다는 뜻이다.

persist=True는 다음 프레임에서도 같은 사람에게 같은 **track_id**를 유지하려는 설정이다.

모든 프레임을 저장하지 않는다. 같은 사람의 사진을 일정 시간 간격으로 저장하고, 한 track에서 너무 많은 사진이 나오지 않도록 제한한다.

```
minimum_gap = max(1, round(fps * sample_every_seconds))

if frame_index - last_saved_frame.get(track_id, -minimum_gap) < minimum_gap:
    continue
if saved_per_track[track_id] >= max_crops_per_track:
    continue
```

이렇게 하면 같은 사람이 10번 화면에 나왔다고 중앙 서버에 10번 후보를 보내지 않고, 사람별 대표 사진을 모아 한 후보로 처리할 수 있다.

9.4 작은 사진·잘린 사진 버리기

사람이 화면에서 너무 작거나 몸이 거의 잘렸으면 색상과 머리 모양을 제대로 볼 수 없다. **crop_quality()**는 사진의 가로·세로 크기를 보고 품질 점수를 만든다.

```
quality = crop_quality(frame.crop_path)
if quality < minimum_person_crop_quality:
    continue
```


좋지 않은 사진을 억지로 모델에 넣으면 모델이 배경이나 픽셀 노이즈를 사람 특징으로 착각할 수 있다.

9.5 각 모델이 후보를 검사하기

실제 **analyze()**의 핵심 흐름은 다음과 같다.

```
semantic = _contrastive_clip_scores(
    frames,
    prompt,
    exclusion_prompt,
    clip,
    device,
)
color = _track_color_values(frames, attributes)
fine = fine_runtime.score(frames) if fine_runtime else {}
identity = score_solider(
    frames,
    identity_anchor,
    config,
    solider_encoder,
)
par = solider_par.score(frames, solider_encoder) if solider_par else {}
```

각 결과의 뜻은 다음과 같다.

코드 결과	쉬운 뜻
semantic	입력한 글과 사진이 얼마나 비슷한가
color	상의·하의 색이 맞는가
fine	세부 속성 head가 판단한 값
identity	기준 사진과 같은 사람처럼 보이는가
par	사람 속성 분류 head가 판단한 값

어떤 모델이 꺼져 있어도 전체 시스템이 바로 멈추지 않는다. 없는 결과는 “없음”으로 남기고, 실제 있는 결과만 다음 단계에 사용한다.

9.6 여러 프레임을 사람 한 명의 결과로 합치기

한 프레임만 보면 눈을 감거나 몸이 가려질 수 있다. 그래서 같은 **track_id**에 속한 사진 중 점수가 높은 대표 사진을 골라 평균을 낸다.

```
base = aggregate_track_scores(
    frame_rows,
    attributes,
    top_frames=3,
)
base = add_track_consistency(base, frames)
```

top_frames=3은 모든 사진을 똑같이 믿지 않고, 가장 잘 보이는 세 장을 중심으로 사람 단위 결과를 만들겠다는 뜻이다.

9.7 결과 합치기: late fusion

각 모델의 점수를 마지막에 합치는 함수는 **fuse_track_scores()**다.

```
signals = [
    (scores.semantic, 0.16),
    (scores.temporal, 0.06),
    (scores.spatial, 0.04),
    (scores.quality, 0.04),
]
if scores.required_color is not None:
    signals.append((scores.required_color, 0.18))
if scores.identity is not None:
    signals.append((scores.identity, 0.28))
if historical is not None:
    signals.append((historical, 0.16))
if qwen is not None:
    signals.append((qwen, 0.12))

total_weight = sum(weight for _, weight in signals)
score = sum(value * weight for value, weight in signals) / total_weight
```

가중치는 "정답 확률"이 아니라 여러 증거를 합칠 때의 중요도다. 예를 들어 identity 사진이 없으면 identity 점수를 0점으로 만들지 않는다. 없는 증거는 계산에서 빼고, 있는 증거의 비율만 다시 계산한다. 이것을 재정규화라고 한다.

9.8 Qwen은 상위 후보만 자세히 보기

Qwen이 영상의 모든 프레임을 보면 시간이 오래 걸린다. 먼저 CLIP·색상·SOLIDER로 후보 순위를 만들고, 그중 상위 **top_k**개만 Qwen에게 보낸다.

```
review_order = sorted(
    evidence_frame_by_track,
    key=lambda track_id: fused_by_track[track_id].score,
    reverse=True,
)[: self.qwen_review.top_k]

for track_id in review_order:
    representative = evidence_frame_by_track[track_id]
    review, status = self.qwen_review.review(
        representative.crop_path,
        case_id=request.case_id,
        prompt=request.prompt,
    )
```

Qwen이 낸 점수가 있으면 다시 점수를 합친다.

```
fused = fuse_track_scores(
    base,
    historical=historical_score,
    qwen=qwen_score,
)
```

즉 Qwen은 처음부터 모든 일을 하는 모델이 아니라, 다른 모델들이 좁힌 후보를 자세히 설명하는 마지막 보조 담당자다.

9.9 후보로 등록할지 결정하기

```
decision = decide_track(
    fused,
    attributes,
    minimum_output_score=minimum_output_score,
    color_reject_threshold=color_reject_threshold,
    similarity_threshold=request.similarity_threshold,
)
```

다음 상황에서는 후보로 내보내지 않는다.

- 실종자 설명에 색이 있는데 색상 점수가 너무 낮음
- 서버가 보낸 기준값보다 점수가 낮음
- 워커의 최소 후보 점수보다 낮음
- 사람이 보기에 사용할 수 있는 증거가 없음

후보가 되었다고 자동 확정하는 것은 아니다. 결과는 관리자 검토 상태로 남긴다.

10. 중앙 서버로 보내는 결과

현재 코드: **src/qwen_backend/candidate_runtime.py**

후보 하나는 코드에서 **RuntimeCandidate**라는 자료 형태로 표현되며, 다음 정보로 구성된다.

```
{
    "candidateKey": "track-17",
    "frameOffsetMs": 18400,
    "similarity": 0.731204,
    "framePath": "candidates/track-17/frame-018400.jpg",
    "cropPath": "candidates/track-17/crop-018400.jpg",
    "boundingBox": {
        "x": 120,
        "y": 80,
        "width": 300,
        "height": 700
    }
},
```

```
"attributeSummary": "YOLO=used;SOLIDER=used;Qwen=used;top1;operator_review"
}
```

각 항목의 뜻은 다음과 같다.

항목	뜻
candidateKey	후보를 구분하는 이름
frameOffsetMs	영상 시작 후 몇 밀리초에 나왔는가
similarity	여러 증거를 합친 후보 점수
framePath	원본 프레임 위치
cropPath	사람만 잘라낸 사진 위치
boundingBox	영상 안 사람의 위치
attributeSummary	어떤 모델이 사용되었는지 기록

워커는 결과 사진이 **outputDir** 밖으로 빠져나가지 않았는지, 요청한 시간 범위 밖의 후보가 아닌지도 확인한다. 이것은 잘못된 파일이나 다른 작업의 사진이 전송되는 것을 막는 안전장치다.

track_id는 한 영상 안에서 프레임을 묶는 번호일 뿐이다. 카메라가 바뀌어도 같은 사람이라는 것을 자동으로 보장하는 전역 번호는 아니다. 그래서 최종 결과는 **operator_review** 상태로 관리자가 확인한다.

11. 전체 코드 흐름을 한 번에 보기

```
def analyze(request):
    # 1. 영상에서 사람과 track을 찾는다.
    detected = detect_person_tracks(
        request.video_path,
        request.output_dir,
        weights=yolo_weights,
        tracker=tracker,
        device=device,
        confidence=confidence,
        stride=stride,
        sample_every_seconds=sample_every_seconds,
        max_crops_per_track=max_crops_per_track,
        margin=margin,
        search_from_ms=request.search_from_ms,
        search_to_ms=request.search_to_ms,
    )

    # 2. 너무 작거나 잘린 사진을 제외한다.
    frames = [
        frame for frame in detected
        if crop_quality(frame.crop_path) >= minimum_person_crop_quality
    ]

    # 3. 여러 전문가에게 같은 후보 사진을 보여준다.
    scores = score_with_clip_color_solider_and_par(frames, request.prompt)

    # 4. 같은 track의 사진을 한 사람의 결과로 합친다.
    by_track = aggregate_by_track(scores)

    # 5. 높은 후보만 Qwen에게 자세한 설명을 요청한다.
    qwen_scores = review_top_k_with_qwen(by_track, request.prompt)

    # 6. 결과를 다시 합치고, 관리자에게 보여줄 후보만 선택한다.
    final_scores = fuse_track_scores(by_track, qwen=qwen_scores)
    return make_operator_review_candidates(final_scores)
```

마지막 예시는 이해를 위한 축약 코드다. 실제 구현은 **src/qwen_backend/multi_model_candidate_engine.py**에 있으며, 모델이 없을 때의 처리, 파일 경로 확인, 시간 범위 확인, 상태 기록까지 포함한다.

12. 실제 실행 명령

12.1 CLIP hard-label-SOLIDER 증류 학습

```
uv run scripts/train_clip_vitl14_distill.py ₩
--output experiments/results/clip-vit-l14-distillation.json ₩
--data-root experiments/data/pa100k_full ₩
--clip-checkpoint openai/clip-vit-large-patch14 ₩
--teacher-checkpoint experiments/models/solider_swin_base.pth ₩
--train-rows 80000 ₩
--val-rows 10000 ₩
--test-rows 10000 ₩
--distill-alpha 0.35 ₩
--temperature 2.0
```

12.2 승인된 Sonnet 기록을 Qwen 자료로 만들기

```
uv run python -m qwen_backend.distillation_cli validate ₩
--input datasets/candidate/distillation.jsonl ₩
--image-root datasets/candidate/images

uv run python -m qwen_backend.distillation_cli prepare ₩
--input datasets/candidate/distillation.jsonl ₩
--image-root datasets/candidate/images ₩
--output datasets/candidate/qwen_train.jsonl
```

12.3 후보 결과 확인

```
uv run python -m qwen_backend.evaluation_cli ₩
--reference datasets/evaluation/gallery.jsonl ₩
--predictions experiments/results/qwen_predictions.jsonl ₩
--output experiments/results/qwen_report.json
```

학습이 끝났는지 판단할 때는 학습 데이터 점수만 보지 않는다. 학습에 넣지 않은 사람의 영상, 다른 시간대, 다른 카메라, 비슷하게 생긴 방해 인물을 포함한 평가가 필요하다.

13. 현재 코드와 실험 상태

항목	현재 문서에서 확인한 상태
CLIP 속성 head 학습 코드	저장소에 있음
CLIP-SOLIDER logit 증류 코드	저장소에 있음
Sonnet 응답을 속성 정답으로 쓰는 코드	저장소에 있음
Sonnet 기록의 승인-SHA-256 검증	저장소에 있음
SOLIDER ArcFace-Triplet-part 학습 코드	저장소에 있음
YOLO → crop → 모델 분석 → track 집계	저장소에 있음
late fusion과 후보 결정	저장소에 있음
Qwen JSONL 준비 코드	저장소에 있음
Qwen 장시간 LoRA runner	현재 checkout에는 없음
프로젝트 CCTV 전체의 일반화 85% 증명	현재 문서만으로는 확정하지 않음

PA-100K나 공개 proxy 데이터에서 얻은 속성 점수는 실제 프로젝트 CCTV의 동일인 검색 점수와 같은 것이 아니다. 프로젝트에서 모델을 최종 채택하려면 identity가 여러 명이고, 방해 인물도 있으며, 카메라와 시간이 분리된 held-out 영상으로 Rank-1, Recall@5, false match, false reject, review rate와 처리 시간을 함께 측정해야 한다.

14. 마지막으로 한 문장씩 정리

- CLIP은 “이 글과 사진이 비슷한가?”를 본다.

- 색상 모델은 "상위와 하의 색이 맞는가?"를 본다.
- SOLIDER는 "기준 사진과 같은 사람처럼 보이는가?"를 본다.
- Qwen은 "상위 후보의 특징을 말로 설명하면 어떤가?"를 보조적으로 확인한다.
- Sonnet은 학습 전에 속성 답변을 만들어 주는 선생님이다.
- YOLO는 영상에서 사람을 찾아 사진으로 잘라 준다.
- **track_id**는 여러 프레임을 한 사람으로 묶는다.
- late fusion은 각 모델의 증거를 마지막에 합친다.
- 최종 후보는 자동 확정이 아니라 관리자 검토용이다.

코드 위치

```
ai-worker/scripts/train_clip_vitl14_distill.py
ai-worker/scripts/finetune_clip_l14_sonnet_aux.py
ai-worker/scripts/finetune_prid2011_solider_backbone.py
ai-worker/src/qwen_backend/annotation_cli.py
ai-worker/src/qwen_backend/distillation.py
ai-worker/src/qwen_backend/distillation_cli.py
ai-worker/src/qwen_backend/video_tracks.py
ai-worker/src/qwen_backend/attribute_ensemble.py
ai-worker/src/qwen_backend/multi_model_candidate_engine.py
ai-worker/src/qwen_backend/candidate_runtime.py
```